

Базы данных

Лекция 09 – Berkeley DB. Управление файлами БД

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2025

Оглавление

Проектирование приложения для восстановления.....	3
Восстановление для многопоточных приложений.....	3
Восстановление в многопроцессных приложениях.....	7
DB_ENV->set_thread_count().....	13
DB_ENV->set_isalive().....	15
DB_ENV->set_thread_id().....	17
DB_ENV->failchk().....	19
Использование горячих аварийных переключений (Using Hot Failovers).....	22
Удаление файлов журнала (Removing Log Files).....	25
Настройка подсистемы ведения журнала.....	28
Установка размера файла журнала (Setting the Log File Size).....	29
Настройка размера области ведения журнала (Configuring the Logging Region Size).....	31
Настройка ведения журнала в памяти (ACI).....	32
Настройка размера буфера журнала в памяти (Setting In-Memory Log Buffer Size).....	36
Резюме и примеры.....	37
Анатомия транзакционного приложения.....	37
Пример транзакции.....	40
Пример транзакции в памяти.....	62

Проектирование приложения для восстановления

При создании приложения базы данных следует всегда подумать о том, как будет запускаться восстановление. Если это однопоточное приложение с одним процессом, восстановление запустить довольно просто, когда приложение впервые открывает свое окружение.

В этом случае нужно только решить, будет ли запускаться восстановление каждый раз, когда приложение стартует (рекомендуется) или только иногда, запускаемое параметром, контролируемым пользователем приложения.

Однако для многопоточных и многопроцессных приложений необходимо тщательно продумать код запуска приложения, чтобы запускать восстановление только тогда, когда это имеет смысл.

Восстановление для многопоточных приложений

1) Если приложение использует только один дескриптор окружения, то обработка восстановления для многопоточного приложения не сложнее, чем для однопоточного приложения. В этом случае просто открывается окружение в главном потоке приложения, а затем этот дескриптор передается каждому из потоков, которые будут выполнять операции с базой данных. (Пример транзакции, стр. 40).

2) В качестве альтернативы можно сделать так, чтобы каждый рабочий поток открывал свой собственный дескриптор окружения. Однако в этом случае проектирование восстановления немного сложнее.

Обычно, когда поток, выполняющий операции с базой данных, дает сбой или зависает, часто проще всего перезапустить приложение и запустить восстановление в обычном режиме при следующем запуске приложения.

Однако не все приложения могут позволить себе перезапуск из-за неправильного поведения одного потока. Если необходимо продолжить работу в условиях неправильного поведения потока, то, если поток, выполняющий операции с базой данных, выходит из строя или зависает, как минимум, должно быть запущено восстановление.

Восстановление очищает окружение от всех неосвобожденных блокировок, включая любые, которые могут быть принадлежать прерванному потоку.

Если эти блокировки не сбрасываются, другие потоки, выполняющие операции с базой данных, могут столкнуться с блокировками, полученными, но так и не сброшенными невыполненным потоком. В результате все приложение зависнет на неопределенный срок.

Чтобы запустить восстановление в этих обстоятельствах, следует:

1) **приостановить или завершить работу всех других потоков**, выполняющих операции с базой данных;

2) **отказаться от всех открытых дескрипторов окружения** – попытка корректно закрыть эти дескрипторы может привести к проблемам, поскольку состояние окружений при сбое потока может быть неопределенным и закрытие завершится неудачей, если окружение нуждается в восстановлении. По этой причине лучше и проще всего просто от дескриптора отказаться.

3) **открыть новые дескрипторы окружения**, запустив восстановление по мере их открытия. (Обычное восстановление, стр. Ошибка: источник перекрёстной ссылки не найден).

4) перезапустить все потоки базы данных.

Традиционный способ справиться с этой активностью — создать поток-наблюдатель, который следит за тем, что с потоками все в порядке, и выполняет вышеуказанные действия, если это не так.

Однако в случае, когда каждый рабочий поток открывает и поддерживает свой собственный дескриптор окружения, восстановление усложняется по двум причинам:

1) Для некоторых приложений и рабочих нагрузок может быть целесообразно предоставить потокам базы данных возможность аккуратно завершить любые текущие транзакции.

Если это так, код должен быть способен сигнализировать каждому потоку остановить действия базы данных и закрыть его окружение – если же просто запустить восстановление в окружении, потоки базы данных обнаружат это в процессе выполнения своих операций с БД и выйдут из строя.

2) Код должен гарантировать, что только один поток запустит восстановление, прежде чем разрешить всем остальным потокам открыть соответствующие дескрипторы окружения (posix barriers).

3) Восстановление должно быть однопоточным, потому что когда в окружении запускается восстановление, окружение удаляется, а затем создается заново. Это приведет к тому, что все другие процессы и потоки, когда попытаются выполнить операции в недавно восстановленном окружении, «потерпят неудачу».

4) Если все потоки запустят восстановление при запуске, то, скорее всего, некоторые потоки потерпят неудачу, потому что используемое ими окружение было уже восстановлено. Это заставит поток повторно выполнить свой собственный путь восстановления.

В лучшем случае это неэффективно, а в худшем — может привести к тому, что приложение перейдет в бесконечный цикл восстановления.

Восстановление в многопроцессных приложениях

Часто приложения БД используют несколько процессов для взаимодействия с базами данных. Например, может быть долго работающий процесс, такой как какой-то сервер, а затем ряд административных инструментов, которые вы используете для проверки и администрирования базовых баз данных. Или, в некоторых веб-архитектурах, различные службы запускаются как независимые процессы, которые управляются сервером.

В любом случае, восстановление для многопроцессного окружения осложняется двумя причинами:

1) В случае необходимости запуска восстановления *бывает необходимо* уведомить процессы, взаимодействующие с окружением, о том, что вот-вот произойдет восстановление, и дать им возможность корректно завершить работу. Стоит ли это делать, полностью зависит от характера приложения.

Для некоторых долго работающих приложений с несколькими процессами, выполняющих серьезную работу *это делать необходимо*.

Другие приложения с процессами, выполняющими операции с базами данных, которые оказались повреждены из-за ошибок в других процессах, скорее всего это делать не будут, поскольку для этой группы шансы на корректное завершение работы могут быть низкими.

2) В отличие от сценариев с одним процессом, запуск восстановления для каждого процесса, взаимодействующего с базами данных, при его старте будет расточительным. Это отчасти потому, что восстановление занимает некоторое время, но в основном имеет смысл избежать ситуации, когда сервер будет повторно открывать все свои дескрипторы окружения только потому, что кто-то запустил утилиту администрирования базы данных из командной строки, которая всегда запускает восстановление.

DB предлагает два метода, с помощью которых можно управлять восстановлением для многопроцессных приложений DB. Каждый из них имеет свои сильные и слабые стороны.

Эффекты многопроцессного восстановления

Стоит отметить, что процессы восстановления могут привести к тому, что один процесс будет выполнять восстановление, в то время как другие процессы в данный момент активно выполняют операции с базой данных.

Когда это происходит, текущая операция базы данных аварийно завершается сбоем, указывая на состояние **DB_RUNRECOVERY**. Это означает, что приложение должно:

- 1) немедленно прекратить любые операции с базой данных;
- 2) отменить все открытые им дескрипторы окружения;
- 3) получить и открыть новые дескрипторы.

Чистый эффект от этого заключается в том, что любые записи, выполненные «неразрешенными» транзакциями, будут потеряны.

Для постоянных приложений (например, серверов) предоставляемые ими сервисы будут недоступны в течение времени, необходимого для завершения восстановления и для всех участвующих процессов, чтобы повторно открыть свои дескрипторы окружения.

1) Регистрация процесса (Process Registration)

Один из способов управления многопроцессным восстановлением — это «регистрация» каждым процессом своего окружения.

При этом процесс получает возможность видеть, используют ли окружение другие приложения, и если да, то были ли они ненормально завершены. Если обнаружено ненормальное завершение, процесс запускает восстановление, в противном случае не запускает.

Использование регистрации процесса также гарантирует, что восстановление будет сериализовано между приложениями. То есть, только один процесс за раз имеет возможность запустить восстановление. Обычно это означает, что первый запущенный процесс запустит восстановление, а все остальные процессы молча не будут запускать восстановление, поскольку оно не нужно.

Чтобы приложение зарегистрировало свое окружение, следует при открытии окружения указать флаг **DB_REGISTER**. Также можно указать **DB_RECOVER** (нормальное восст.).

В этом случае если во время открытия DB определяет, что необходимо запустить восстановление, она автоматически запустит правильный тип восстановления. Указывать **DB_RECOVER_FATAL** при использовании флага **DB_REGISTER** является ошибкой.

Если окружение будет зарегистрировано, а нормальное восстановление не будет указано, то если процесс регистрации определит необходимость восстановления, оно запущено не будет.

В этом случае открытие окружения просто завершается неудачей, возвращая **DB_RUNRECOVERY**.

Примечание: Если нормальное восстановление при открытии в первый раз зарегистрированного окружения в приложении указано не будет, то это приложение завершит открытие окружения сбоем, вернув **DB_RUNRECOVERY**. Это связано с тем, что первый процесс для регистрации должен создать внутренний файл регистрации, и при создании этого файла принудительно выполняется восстановление.

Чтобы избежать ненормального завершения открытого окружения, следует указать восстановление в открытом окружении хотя бы для первого процесса, запускаемого в приложении.

Кроме того, если при регистрации своего окружения указывается **DB_ENV_FAILCHK**, то при открытии окружения выполняется *проверка на отказ*.

Если во время процесса проверки на отказ обнаруживается ненормальное завершение для любого из процессов, задействованных в приложении, DB снимает все блокировки чтения, удерживаемые мертвым процессом, и выполняет прерывания транзакций по мере необходимости.

Это делается в попытке очистить окружение. В этой ситуации, если общая очистка окружения невозможна и если нормальное восстановление не указано при открытии окружения, то открытие будет прервано с возвращением **DB_RUNRECOVERY**.

Если же такая ситуация возникает и было указано нормальное восстановление, то чтобы вернуть окружение в работоспособное состояние, запускается соответствующий тип восстановления (нормальное или фатальное).

Существуют некоторые ограничения/требования в отношении координации различных процессов при восстановлении с помощью регистрации:

1) Может быть только один дескриптор окружения на одно окружение в одном процессе. В случае многопоточных процессов дескриптор окружения должен быть общим для потоков.

2) Все процессы, совместно использующие окружение, должны использовать регистрацию. Если же регистрация не используется единообразным образом во всех участвующих процессах, то можно получить противоречивые результаты с точки зрения способности приложения распознавать необходимость запуска восстановления.

Проверка отказов (Failure Checking)

Для очень больших и надежных многопроцессных приложений наиболее распространенным способом обеспечения работы всех процессов в соответствии с их предназначением является использование сторожевого процесса, следящим за состоянием процессов.

Чтобы помочь процессу-сторожу, DB предлагает механизм *проверки отказов*.

Когда поток управления выходит из строя с открытыми дескрипторами окружения, результатом является то, что ресурсы могут остаться заблокированными или поврежденными.

Другие потоки управления в зависимости от шаблонов доступа к данным могут как быстро обнаружить, что ресурсы недоступны, так и не обнаружить их вообще.

Механизм проверки отказов БД позволяет сторожевому процессу обнаружить, является ли окружение непригодным для использования в результате сбоя потока управления.

Механизм проверки отказов должен вызываться периодически (например, раз в минуту) из процесса сторожевого таймера.

Если окружение считается непригодным для использования, то процесс сторожевого таймера уведомляется о том, что следует запустить восстановление.

Затем сторожевой процесс должен фактически запустить восстановление. Также сторожевой процесс должен решить, что делать с текущими запущенными процессами перед запуском восстановления. Например, он может попытаться корректно завершить или уничтожить все соответствующие процессы перед запуском восстановления.

Проверка отказов не обязательно должна запускаться из отдельного процесса, хотя концептуально именно так предполагается использование механизма.

Этот же механизм может использоваться в многопоточном приложении, которому требуется поток-сторожевой таймер.

Для использования проверки на отказы необходимо:

1) Предоставить обратный вызов **is_alive()** с помощью **DB_ENV->set_isalive()**.

DB использует этот метод для определения того, активен ли указанный процесс и поток при выполнении проверки на сбой.

2) По возможности предоставить обратный вызов **thread_id()**, который уникально идентифицирует процесс и поток управления. Этот обратный вызов необходим только в том случае, если стандартные функции идентификации процесса и потока для платформы недостаточны для использования при проверке на сбой. Это требуется редко и обычно потому, что идентификаторы потока и/или процесса, используемые системой, не могут вписаться в беззнаковое целое число.

Этот обратный вызов предоставляется с помощью метода **DB_ENV->set_thread_id()**.

3) Периодически вызывать метод **DB_ENV->failchk()**.

Можно делать это либо периодически (например, раз в минуту), либо всякий раз, когда поток управления завершает работу.

Если этот метод определяет, что поток управления завершил работу, удерживая блокировки чтения, эти блокировки автоматически снимаются.

Если поток управления завершил работу с неразрешенной транзакцией, эта транзакция прерывается.

Если существуют какие-либо другие проблемы помимо этих, такие, что окружение необходимо восстановить, метод вернет **DB_RUNRECOVERY**.

DB_ENV->set_thread_count()

```
#include <db.h>

int DB_ENV->set_thread_count(DB_ENV *dbenv, u_int32_t count);
```

Объявляет приблизительное количество потоков в среде базы данных.

Этот метод выделяет ресурсы в окружении для потоков, которые будет использовать приложение.

Если правильно оценить количество потоков которые будет использовать приложение, не удастся, ресурсы приложения закончатся, и будут возвращаться ошибки при попытке приложения запустить слишком много потоков.

DB_ENV->set_thread_count() не устанавливает максимальное количество потоков, но используется для определения размера памяти и политики освобождения блока управления потоками.

DB_ENV->set_thread_count() должен быть вызван до открытия окружения базы данных. Кроме того, этот метод должен использоваться вместе с методом DB_ENV->failchk().

Если процесс вызывает этот метод без использования DB_ENV->failchk(), программа может не успеть выделить блок управления потоками. Это справедливо для автономных служебных программ Berkeley DB.

Если процесс не настроил функцию is_alive() из метода DB_ENV->set_isalive(), а затем пытается присоединиться к окружению базы данных, настроенной на проверку сбоя с помощью DB_ENV->failchk(), DB_ENV->set_thread_id(), DB_ENV->set_isalive() и DB_ENV->set_thread_count(), программа может не выделить блок управления потоками и не присоединиться к окружению.

Это справедливо для автономных утилит Berkeley DB. Чтобы избежать проблем при использовании автономных утилит Berkeley DB со средами, настроенными на проверку сбоев, следует включить функциональность утилиты непосредственно в приложение или вызвать метод `DB_ENV->failchk( )` перед запуском утилиты.

Количество потоков в среде базы данных также можно настроить с помощью файла `DB_CONFIG` среды исполнения.

Синтаксис записи в этом файле представляет собой одну строку со строчкой `"set_thread_count"`, одним или несколькими пробелами и количеством потоков.

Поскольку файл `DB_CONFIG` считывается при открытии окружения базы данных, он автоматически переопределяет конфигурацию, выполненную до этого момента.

`DB_ENV->set_thread_count( )` настраивает операции, выполняемые с использованием указанного дескриптора `DB_ENV`, а не все операции, выполняемые в окружении базы данных.

`DB_ENV->set_thread_count( )` не может быть вызван после вызова метода `DB_ENV->open( )`.

`DB_ENV->set_thread_count( )` возвращает ненулевое значение ошибки в случае ошибки и 0 в случае успешного выполнения.

Параметры

`count` — приблизительное количество потоков для окружения базы данных.

Ошибки

`DB_ENV->set_thread_count( )` может завершиться ошибкой и вернуть:

`EINVAL` — если метод был вызван после вызова `DB_ENV->open()` или если было указано недопустимое значение флага или параметра.

DB_ENV->set_isalive()

```
#include <db.h>

int DB_ENV->set_isalive(DB_ENV *dbenv,
                       int (*is_alive)(DB_ENV *dbenv,
                                       pid_t pid,
                                       db_threadid_t tid,
                                       u_int32_t flags));
```

Объявляет функцию, которая возвращает значение, если поток управления (истинный поток или процесс) всё ещё выполняется.

DB_ENV->set_isalive() поддерживает метод DB_ENV->failchk().

DB_ENV->set_isalive() настраивает операции, выполняемые с использованием указанного дескриптора DB_ENV, а не все операции, выполняемые в окружении основной базы данных.

DB_ENV->set_isalive() может быть вызван в любой момент жизненного цикла приложения.

DB_ENV->set_isalive() возвращает ненулевое значение ошибки в случае ошибки и 0 в случае успешного выполнения.

Параметры

is_alive — это функция, которая возвращает *ненулевое* значение, если поток управления, идентифицированный аргументами *pid* и *tid*, всё ещё выполняется. Функция принимает четыре аргумента:

dbenv — это дескриптор окружения, в котором выполняются процессы в БД.

pid — идентификатор процесса, возвращаемый функцией, указанной в методе DB_ENV->set_thread_id().

tid — идентификатор потока, возвращаемый функцией, указанной в методе DB_ENV->set_thread_id().

flags — параметр flags должен быть установлен в 0 или иметь следующее значение:

DB_MUTEX_PROCESS_ONLY — возвращает только если активен процесс, идентификатор потока в этом случае следует игнорировать.

Ошибки

Метод DB_ENV->set_isalive() может завершиться ошибкой и вернуть одну из следующих ненулевых ошибок:

EINVAL — указано недопустимое значение флага или параметра.

DB_ENV->set_thread_id()

```
#include <db.h>

int DB_ENV->set_thread_id(DB_ENV *dbenv,
                          void (*thread_id)(DB_ENV *dbenv,
                                              pid_t *pid,
                                              db_threadid_t *tid));
```

Объявляет функцию, возвращающую уникальную пару идентификаторов для текущего потока управления.

DB_ENV->set_thread_id() поддерживает метод DB_ENV->failchk().

DB_ENV->set_thread_id() настраивает операции, выполняемые с использованием указанного дескриптора DB_ENV, а не все операции, выполняемые в основном окружении базы данных.

DB_ENV->set_thread_id() может быть вызван в любой момент жизненного цикла приложения.

DB_ENV->set_thread_id() возвращает ненулевое значение ошибки в случае ошибки и 0 в случае успешного выполнения.

Параметры

thread_id — функция, возвращающая уникальную пару идентификаторов для потока управления в приложении Berkeley DB. Функция принимает три аргумента:

dbenv — дескриптор окружения включающей базы данных.

`pid` — указывает на область памяти с типом `pid_t` или `NULL`. В этой области памяти, если `pid` не равен `NULL` может быть возвращен идентификатор процесса, в котором выполняется текущий поток управления.

`tid` — указывает на область памяти с типом `db_threadid_t` или `NULL`. В этой области памяти, если `tid` не равен `NULL` может быть возвращен идентификатор текущего потока управления.

Ошибки

Метод `DB_ENV->set_thread_id()` может завершиться ошибкой и вернуть одну из следующих ненулевых ошибок:

EINVAL — Указано недопустимое значение флага или параметра.

DB_ENV->failchk()

```
#include <db.h>

int DB_ENV->failchk(DB_ENV *dbenv, u_int32_t flags);
```

The `DB_ENV->failchk()` method checks for threads of control (either a true thread or a process) that have exited while manipulating Berkeley DB library data structures, while holding a logical database lock, or with an unresolved transaction (that is, a transaction that was never aborted or committed). For more information, see *Architecting Data Store and Concurrent Data Store applications*, and *Architecting Transactional Data Store applications*, both in the *Berkeley DB Programmer's Reference Guide*.

The `DB_ENV->failchk()` method is used in conjunction with the `DB_ENV->set_thread_count()` (page 330), `DB_ENV->set_isalive()` (page 317) and `DB_ENV->set_thread_id()` (page 332) methods. Before calling the `failchk()` method, applications must:

Метод `DB_ENV->failchk()` проверяет наличие потоков управления (либо реальных потоков, либо процессов), которые завершились во время работы со структурами данных библиотеки Berkeley DB, удерживая логическую блокировку базы данных или с неразрешённой транзакцией (то есть транзакцией, которая не была ни прервана, ни зафиксирована).

`DB_ENV->failchk()` используется совместно с методами `DB_ENV->set_thread_count()`, `DB_ENV->set_isalive()` и `DB_ENV->set_thread_id()`.

Перед вызовом метода `failchk()` приложения должны:

- 1) Настроить свою базу данных с помощью метода `DB_ENV->set_thread_count()`.
- 2) Установить функцию `is_alive()` и вызвать `DB_ENV->set_isalive()` с этой функцией в качестве параметра `is_alive`.

3) Установить функцию `thread_id( )` и вызвать `DB_ENV->set_thread_id( )` с этой функцией в качестве параметра `thread_id`.

Если какой-либо из этих методов пропущен, программа не сможет выделить блок управления потоком. Это справедливо для автономных утилит Berkeley DB.

Чтобы избежать проблем при использовании автономных утилит Berkeley DB с окружениями, настроенными на проверку сбоев, следует включить функциональность утилиты непосредственно в приложение или вызвать метод `DB_ENV->failchk( )` вместе со связанными с ним методами перед запуском утилиты.

Если метод `DB_ENV->failchk( )` определяет, что поток управления завершился, удерживая блокировки чтения базы данных, он снимает эти блокировки.

Если метод `DB_ENV->failchk( )` определяет, что поток управления завершился с неразрешённой транзакцией, транзакция будет прервана.

В любом из этих случаев метод `DB_ENV->failchk( )` вернёт 0, и приложение сможет продолжить использование среды базы данных.

В любом из этих случаев метод `DB_ENV->failchk( )` также сообщит идентификаторы процесса и потока, связанные с любыми снятыми блокировками или прерванными транзакциями. Эта информация выводится в указанный канал вывода (`DB_ENV->set_msgfile( )`) или передаётся в функцию обратного вызова приложения (`DB_ENV->set_msgcall( )`).

Если метод `DB_ENV->failchk( )` определяет, что поток управления завершился и требуется восстановление окружения базы данных, он возвращает `DB_RUNRECOVERY`.

В этом случае приложение не должно продолжать использовать среду базы данных.

В многопроцессных приложениях рекомендуется, чтобы дескриптор `DB_ENV`, используемый для вызова метода `DB_ENV->failchk( )`, не был общим и, следовательно, не поддерживал совместного использования.

Метод DB_ENV->failchk() не может быть вызван приложением до вызова метода DB_ENV->open().

Метод DB_ENV->failchk() возвращает ненулевое значение ошибки в случае сбоя и 0 в случае успешного выполнения.

Параметры

flags — в настоящее время не используется и должен быть установлен в 0.

Ошибки

Метод DB_ENV->failchk() может завершиться ошибкой и вернуть одну из следующих ненулевых ошибок:

EINVAL — указано недопустимое значение флага или параметра.

Использование горячих аварийных переключений (Using Hot Failovers)

Для целей отказоустойчивости можно поддерживать резервную копию.

Горячие аварийные переключения отличаются от процедур резервного копирования и восстановления тем, что данные, используемые для традиционных резервных копий, обычно копируются в автономное хранилище. Время восстановления для традиционной резервной копии определяется:

1) Насколько быстро можно извлечь этот носитель. Обычно носитель для критических резервных копий перемещается в безопасное место в удаленном месте, поэтому этот шаг может занять относительно много времени.

2) Насколько быстро можно считать резервную копию с носителя на локальный диск. Если очень большие резервные копии или носитель очень медленный, это может быть длительным процессом.

3) Сколько времени потребуется, чтобы запустить катастрофическое восстановление с использованием только что восстановленной резервной копии. Этот процесс может быть длительным, поскольку в процессе восстановления необходимо проверить каждый файл журнала.

При использовании горячего аварийного переключения резервная копия сохраняется в месте, к которому можно достаточно быстро получить доступ. Обычно это второй локальный диск машины. В этой ситуации время восстановления очень быстрое, поскольку требуется только заново открыть окружение и базу данных.

Горячие переключения не защищают от действительно катастрофических бедствий типа пожара, поскольку резервная копия находится на локальной машине. Однако можно защититься от более обыденных проблем, типа поломки жесткого диска, сохраняя резервную копию на втором диске, который управляется альтернативным контроллером диска.

Для поддержания горячего переключения следует:

1) **Скопировать** все активные файлы базы данных в резервный каталог. Можно использовать утилиту командной строки **db_archive** с опцией **-s**, чтобы определить все активные файлы базы данных.

2) Определить все *неактивные файлы журналов* в рабочем окружении и **переместить** их в резервный каталог. Можно использовать команду **db_archive** без опций командной строки, чтобы получить список этих файлов журналов.

3) Определить *активные файлы журналов* в рабочем окружении и **скопировать** их в резервный каталог. Можно использовать команду **db_archive** с опцией **-l**, чтобы получить список этих файлов журналов.

4) Запустить катастрофическое восстановление для резервного каталога. Можно использовать команду **db_recover** с опцией **-c**, чтобы это сделать.

5) При желании скопировать резервную копию в архивное местоположение.

После выполнения этой процедуры можно поддерживать активную горячую резервную копию, повторяя шаги 2–5 так часто, как того требует приложение.

Примечание

Если выполняется шаг 1, за ним должны следовать шаги 2–5, чтобы обеспечить согласованность горячей резервной копии.

Примечание

Вместо использования предыдущей процедуры можно использовать утилиту командной строки **db_hotbackup**, чтобы сделать то же самое. Эта утилита (необязательно) запустит контрольную точку, а затем скопирует все необходимые файлы в целевой каталог.

Чтобы фактически выполнить отказоустойчивость, просто следует:

1) Завершить все процессы, которые работают в исходном окружении.

2) Если есть архивная копия резервного окружения, можно дополнительно попробовать скопировать оставшиеся файлы журналов из исходного окружения и запустить катастрофическое восстановление в этой резервном окружении. Делать это следует только в том случае, если есть архивная копия резервного окружения.

Этот шаг может позволить восстановить данные, созданные или измененные в исходном окружении, но которые не успели отразиться в окружении горячего резервного копирования.

3) Повторно открыть окружение и базы данных в обычном режиме, но использовать вместо рабочего окружения резервное.

Удаление файлов журнала (Removing Log Files)

По умолчанию DB файлы журнала не удаляет. По этой причине файлы журнала DB со временем разрастутся и займут неоправданно большой объем дискового пространства.

Чтобы защититься от этого, следует периодически предпринимать административные действия по удалению файлов журнала, которые больше не используются приложением.

Можно удалить файл журнала, если выполняются все следующие условия:

- файл журнала не участвует в активной транзакции.
- после создания файла журнала была выполнена контрольная точка.
- файл журнала не является единственным файлом журнала в окружении.
- файл журнала, который предполагается удалить, уже был включен в автономную или горячую резервную копию.

Несоблюдение этого последнего условия может привести к тому, что резервные копии станут непригодными для использования.

DB предоставляет несколько механизмов для удаления файлов журналов, которые соответствуют всем критериям, кроме последнего – DB не может узнать, какие файлы журналов уже включены в резервную копию.

Следующие механизмы упрощают удаление ненужных файлов журналов, но могут привести к тому, что резервная копия станет непригодной для использования, если файлы журналов предварительно не будут сохранены в архивном месте.

Все следующие механизмы автоматически удаляют ненужные файлы журналов:

- 1) Запуск утилиты командной строки **db_archive** с опцией **-d**.
- 2) Вызов из приложения **DB_ENV->log_archive()** с флагом **DB_ARCH_REMOVE**.
- 3) Вызов метода **DB_ENV->log_set_config()** с флагом **DB_LOG_AUTO_REMOVE**.

Флаг **DB_LOG_AUTO_REMOVE** можно установить в любой момент жизненного цикла приложения. Установка этого параметра влияет на все дескрипторы окружения, открытые для окружения, а не только на дескриптор, используемый при установке флага.

В отличие от других механизмов удаления журналов, метод 3) фактически приводит к постоянному удалению файлов журналов по мере того, как они становятся ненужными. Это очень удобно если необходимо использовать минимум дискового пространства.

Данный механизм оставит файлы журналов, необходимые для запуска нормального восстановления. Однако весьма вероятно, что этот механизм не позволит запустить катастрофическое восстановление.

НЕ следует использовать этот механизм, если необходимо иметь возможность выполнять катастрофическое восстановление или иметь возможность поддерживать горячее резервное копирование.

Чтобы безопасно удалить файлы журнала и по-прежнему иметь возможность выполнять катастрофическое восстановление, следует использовать утилиту **db_archive**:

1) Следует запустить обычное или горячее резервное копирование (Процедуры резервного копирования, стр. Ошибка: источник перекрёстной ссылки не найден) и, прежде чем продолжить, убедиться, что все эти данные надежно сохранены на резервном носителе.

2) Следует выполнить контрольную точку для получения дополнительной информации, если еще этого не было сделано (Контрольные точки, стр. Ошибка: источник перекрёстной ссылки не найден).

3) Если поддерживается горячее резервное копирование, следует выполнить процедуру горячего резервного копирования (Использование горячих отказов, стр. 22).

4) Следует запустить утилиту **db_archive** с параметром **-d** для рабочего окружения.

5) Запустить утилиту **db_archive** с параметром **-d** для отказоустойчивого окружения,

если оно поддерживается.

Настройка подсистемы ведения журнала

Можно настроить следующие аспекты подсистемы ведения журнала:

- 1) Размер файлов журнала.
- 2) Размер области подсистемы ведения журнала.
- 3) Хранить журналы полностью в памяти.
- 4) Размер буфера журнала в памяти.
- 5) Расположение файлов журнала на диске.

Установка размера файла журнала (Setting the Log File Size)

Всякий раз, когда в файл журнала записывается predetermined объем данных (по умолчанию 10 МБ), DB прекращает использование текущего файла журнала и начинает запись в новый файл.

Можно изменить максимальный объем данных, содержащихся в каждом файле журнала, с помощью метода `DB_ENV->set_lg_max( )`. Этот метод можно использовать в любое время в течение жизненного цикла приложения.

Установка размера файла журнала больше значения по умолчанию — это в основном вопрос удобства и отражение предпочтений приложения в отношении носителя и частоты резервного копирования.

Однако, если будет установлен слишком малый размер файла журнала по сравнению с моделями трафика приложения, можно нажить себе неприятности.

С точки зрения производительности, установка размера файла журнала в низкое значение может привести к тому, что активные транзакции будут приостанавливать свои действия по записи чаще, чем это произошло бы с файлами журнала большего размера.

Всякий раз, когда транзакция завершается, буфер журнала сбрасывается на диск. Обычно другие транзакции могут продолжать писать в буфер журнала, пока выполняется этот сброс. Однако, когда один файл журнала закрывается и создается другой, все транзакции прекращают запись в буфер журнала до тех пор, пока переключение не будет завершено.

Помимо проблем с производительностью, использование меньших файлов журнала может привести к использованию большего количества физических файлов на диске. В результате приложение может исчерпать порядковые номера журнала, в зависимости от того, насколько загружено приложение.

Каждый файл журнала идентифицируется 10-значным номером. Более того, максимальное количество файлов журнала, которые приложение может создать за время своего существования, составляет 2 000 000 000.

Например, если приложение выполняет 6000 транзакций в секунду в течение 24 часов в сутки, и вы регистрируете 500 байт данных за транзакцию в файлах журнала размером 10 МБ, то файлы журнала закончатся примерно через 221 год:

Однако, если приложение записывает 2000 байт данных за транзакцию и использует файлы журнала размером 1 МБ, то файлы журнала закончатся примерно через 5 лет.

Все эти временные рамки, конечно, довольно длительные, но если закончатся файлы журналов после, скажем, 5 лет непрерывной работы, то придется сбросить порядковые номера журналов. Для этого необходимо:

1. Создать резервную копию баз данных, как будто готовясь к катастрофическому сбою.
 2. Сбросить порядковый номер файла журнала с помощью параметра **-r** утилиты **db_load**.
 3. Удалить все файлы журнала из окружения.
- Это единственная ситуация, в которой все файлы журнала удаляются из окружения; во всех остальных случаях сохраняется по крайней мере один файл журнала.
4. Перезапустить приложение.

Настройка размера области ведения журнала (Configuring the Logging Region Size)

Размер области подсистемы ведения журнала по умолчанию составляет 60 КБ. Область ведения журнала используется для хранения имен файлов, поэтому может потребоваться увеличить ее размер, если будет открыто и зарегистрировано в менеджере журналов БД большое количество файлов (в случае очень большого количества баз данных).

Можно задать размер области ведения журнала с помощью метода **DB_ENV->set_lg_regionmax()**.

Этот метод можно вызвать только до открытия первого дескриптора окружения для приложения.

Настройка ведения журнала в памяти (ACI)

Можно настроить подсистему ведения журналов таким образом, чтобы журналы хранились полностью в памяти.

Однако, следствием этого является отказ от гарантии транзакционной устойчивости – без файлов журналов нет возможности запустить восстановление, поэтому любые системные или программные сбои, которые могут возникнуть, могут повредить базы данных.

Тем не менее, отказавшись от гарантий долговечности, можно значительно улучшить пропускную способность приложения, избежав дискового ввода-вывода, необходимого для записи информации журнала на диск. В этом случае все равно транзакционные атомарности, согласованности и гарантии изоляции сохраняются.

Чтобы настроить подсистему журналирования для сохранения журналов полностью в памяти, следует:

- Убедиться, что буфер журнала способен хранить всю информацию журнала, которая может накапливаться во время самой длительной транзакции.
- Не следует при открытии окружения запускать обычное восстановление. В этой конфигурации нет доступных файлов журнала, по которым можно запустить восстановление. В результате, если при открытии окружения будет указано восстановление, оно будет проигнорировано.
- Указать **DB_LOG_IN_MEMORY** для метода **DB_ENV->log_set_config()**. Это следует указать до того, как приложение откроет свой первый дескриптор окружения.

Ниже пример


```
#include <stdio.h>
#include <stdlib.h>
#include "db.h"

int main(void) {

    int ret, ret_c;
    u_int32_t db_flags, env_flags;
    DB_ENV *envp = NULL;
    const char *db_home_dir = "/tmp/myEnvironment";

    // Создание окружения
    ret = db_env_create(&envp, 0);
    if (ret != 0) {
        fprintf(stderr, "Ошибка создания дескриптора окружения: %s\n",
                db_strerror(ret));
        return (EXIT_FAILURE);
    }

    // Указываем, что журналирование должно производиться только в памяти.
    // Это означает, что мы отказываемся от гарантии надежности транзакций.
    envp->log_set_config(envp, DB_LOG_IN_MEMORY, 1);

    // Настраиваем размер буфера памяти журнала. Он должен быть достаточно
    // большим, чтобы вместить всю информацию для самой длительной транзакции.
    // Если журналирование выполняется в памяти, размер буфера журнала по
    // умолчанию составляет 1 МБ.
```

```
// В этом примере мы произвольно устанавливаем буфер журнала равным 5 МБ.
ret = envp->set_lg_bsize(envp, 5 * 1024 * 1024);
if (ret != 0) {
    fprintf(stderr, "Error setting log buffer size: %s\n",
        db_strerror(ret));
    goto err;
}
// Устанавливаем обычные флаги для транзакционной подсистемы.
// Мы НЕ указываем DB_RECOVER. Также следует помнить, что подсистема
// журналирования автоматически включается при инициализации транзакционной
// подсистемы, поэтому мы явно не включаем здесь журналирование.
env_flags =
    DB_CREATE |      // Создать окружение, если не существует
    DB_INIT_LOCK |   // Инициализация блокировок
    DB_INIT_LOG |    // Инициализация журналирования
    DB_INIT_MPOOL |  // Инициализация кеширования в памяти
    DB_THREAD |      // Free-thread the env handle
    DB_INIT_TXN;     // Инициализация транзакций

// Открываем окружение как обычно
ret = envp->open(envp, db_home_dir, env_flags, 0);
if (ret != 0) {
    fprintf(stderr, "Ошибка открытия окружения: %s\n",
        db_strerror(ret));
    goto err;
}
```

```
    // Создаем транзакции и выполняем операции с базой данных.
    // Эта часть опущена для краткости.
    ...
err:
    // Закрываем базы данных (опущено)
    ...
    // Закрываем окружение
    if (envp != NULL) {
        ret_c = envp->close(envp, 0);
        if (ret_c != 0) {
            fprintf(stderr, "environment close failed: %s\n",
                        db_strerror(ret_c));
            ret = ret_c;
        }
    }
    return (ret == 0 ? EXIT_SUCCESS : EXIT_FAILURE);
}
```

Настройка размера буфера журнала в памяти (Setting In-Memory Log Buffer Size)

Если приложение настроено на ведение журнала на диске (поведение по умолчанию для транзакционных приложений), информация журнала хранится в памяти до тех пор, пока не заполнится выделенное для этого пространство или транзакция не заставит записать информацию журнала на диск.

Можно увеличить объем памяти, доступной для буфера журнала файла. Это повышает пропускную способность для длительных транзакций или для транзакций, которые производят большой объем данных.

Если подсистема ведения журнала настроена на сохранение журнала целиком в памяти, очень важно настроить размер буфера журнала, поскольку буфер журнала должен быть способен хранить всю информацию журнала, которая может накапливаться в течение самой длительной транзакции.

Следует убедиться, что размер буфера журнала в памяти достаточно велик, чтобы ни одна транзакция не охватывала весь буфер. Также следует избегать состояния, когда буфер в памяти заполнен и не может быть освобождено место, поскольку транзакция, которая начала первый «файл» журнала, все еще активна.

Если подсистема ведения журнала настроена на ведение журнала на диске, размер буфера журнала по умолчанию составляет 32 КБ. Если настроено ведение журнала в памяти, размер буфера журнала по умолчанию составляет 1 МБ.

Можно увеличить размер буфера журнала с помощью метода `DB_ENV->set_lg_bsize( )`. Этот метод можно вызвать только до открытия первого дескриптора окружения для приложения.

Резюме и примеры

В этом руководстве мы представили концепции и механизмы, которые нужны для обеспечения транзакционной защиты приложения. В этой главе мы суммируем эти механизмы и предоставим полный пример многопоточного транзакционного приложения БД.

Анатомия транзакционного приложения

Транзакционные приложения характеризуются выполнением следующих действий:

- 1) Создание дескриптора окружения.
- 2) Открытие окружения, указав, что будут использоваться следующие подсистемы:
 - Транзакционная подсистема (это также инициализирует подсистему ведения журнала).
 - Пул памяти (кэш в памяти).
 - Подсистема ведения журнала.
 - Подсистема блокировки (если приложение является многопроцессным или многопоточным).

Также настоятельно рекомендуется запустить обычное восстановление при первом открытии окружения.

Обычное восстановление проверяет только те журналы, которые необходимы для обеспечения согласованности файлов базы данных относительно информации, содержащейся в файлах журналов.

3) При желании следует запустить любые служебные потоки, которые могут понадобиться. Служебные потоки можно использовать для периодического запуска контрольных точек или для периодического запуска детектора взаимоблокировок, если встроенный детектор взаимоблокировок БД не используется.

4) Откройте любые необходимые дескрипторы базы данных.

5) Запустите рабочие потоки. Сколько из них нужно и как они разделяют свою рабочую нагрузку БД, полностью зависит от требований приложения. Однако любые рабочие потоки, которые выполняют операции записи, будут делать следующее:

a. Начать транзакцию.

b. Выполнить одну или несколько операций чтения и записи.

c. Зафиксировать транзакцию, если все прошло хорошо.

d. Прервать и повторить операцию, если обнаружена взаимоблокировка.

e. Прервать транзакцию для большинства других ошибок.

б) При завершении работы приложения:

a. Убедиться, что нет открытых курсоров.

b. Убедиться, что нет активных транзакций. Либо отменить, либо зафиксировать все транзакции перед завершением работы.

c. Закрыть базы данных.

d. Закрыть окружение.

Примечание

Надежные приложения БД должны контролировать свои рабочие потоки, чтобы убедиться, что они не прекратились неожиданно. Если поток завершается ненормально, следует остановить все рабочие потоки, а затем запустить нормальное восстановление (для этого придется заново открыть окружение). Это единственный способ очистить любые ресурсы (например, блокировку или мьютекс), которые мог удерживать ненормально завершающийся рабочий поток в момент своей остановки.

Невыполнение этого восстановления может привести к тому, что все еще работающие рабочие потоки в конечном итоге будут заблокированы навсегда в ожидании блокировки, которая никогда не будет снята.

В дополнение к этим действиям, которые полностью обрабатываются кодом в приложении, есть некоторые административные действия, которые следует выполнить:

- Следует периодически создавать контрольные точки для приложения. Контрольные точки сократят время выполнения восстановления в случае необходимости. (Контрольные точки, стр. Ошибка: источник перекрёстной ссылки не найден).

- Следует периодически создавать резервные копии базы данных и файлов журналов. Это необходимо для того, чтобы в полной мере получить гарантию долговечности, предоставляемую поддержкой транзакций ACID в базе данных. (Процедуры резервного копирования, стр. Ошибка: источник перекрёстной ссылки не найден).

- Можно захотеть поддерживать горячее аварийное переключение, если важна круглосуточная обработка с быстрым перезапуском в случае сбоя диска. (Использование горячих аварийных переключений, стр. 22).

Пример транзакции

Следующий код представляет собой полностью функциональный пример многопоточного транзакционного приложения БД. Для улучшения переносимости между платформами этот пример использует pthreads для поддержки потоков.

Пример открывает окружение и базу данных, а затем создает 5 потоков, каждый из которых записывает 500 записей в базу данных. Ключи, используемые для этих записей, представляют собой predetermined строки, в то время как данные представляют собой случайное значение. Это означает, что фактические данные являются произвольными и, следовательно, неинтересными. Такой подход выбран только потому, что для его реализации требуется минимум кода, и данные не будут мешать основным моментам этого примера.

Каждый поток записывает 10 записей в рамках одной транзакции перед фиксацией и записью еще 10 (это повторяется 50 раз). В конце каждой транзакции, но перед фиксацией, каждый поток вызывает функцию, которая использует курсор для чтения каждой записи в базе данных. Это происходит для того, чтобы сделать некоторые замечания относительно чтения базы данных в транзакционном окружении.

Каждый поток записи выполняет обнаружение взаимоблокировок. Кроме того, при открытии окружения выполняется обычное восстановление.

Все начинается с подключения заголовков

```
/* File: txn_guide.c */
/* We assume an ANSI-compatible compiler */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <db.h>
#ifdef _WIN32
extern int getopt(int, char * const *, const char *);
#else
#include <unistd.h>
#endif
```

Также нужна директива, которая используется для количества потоков, которые должна создать программа:

```
/* Run 5 writers threads at a time. */
#define NUMWRITERS 5
```

Далее объявляются пара глобальных переменных, используемых потоками, а также используемые функции.

```
// Печать значений типа pthread_t зависит от реализации, поэтому создаются
// собственные идентификаторы потоков для вывода.
int          global_thread_num;
pthread_mutex_t thread_num_lock;

// Forward declarations
int count_records(DB *, DB_TXN *);
int open_db(DB **, const char *, const char *, DB_ENV *, u_int32_t);
int usage(void);
void* writer_thread(void *);
```

Функция использования показывает единственный параметр командной строки:

```
/* Usage function */
int usage() {
    fprintf(stderr, " [-h <database_home_directory>]\n");
    return (EXIT_FAILURE);
}
```

Вспомогательный код завершен.

Итак, основная функция программы.

Сначала мы объявляем все переменные и инициализируем дескрипторы БД.

```
int main(int argc, char *argv[]) {

    //Инициализируем дескрипторы
    DB      *dbp  = NULL;
    DB_ENV  *envp = NULL;

    // массив учета пишущих потоков
    pthread_t writer_threads[NUMWRITERS];

    int ch, i, ret, ret_t;
    u_int32_t env_flags;
    char *db_home_dir;

    //Имя приложения
    const char *prog_name = "txn_guide";

    // Имя файла базы данных
    const char *file_name = "mydb.db";
```

Теперь нужно разобрать командную строку — узнать, где находится каталог окружения. Если при запуске не указана опция -h, используется текущий рабочий каталог.

```
    // Parse the command line arguments
#ifdef _WIN32                // __MINGW32__
    db_home_dir = ".\\\";
#else
    db_home_dir = "./\";
#endif
    while ((ch = getopt(argc, argv, "h:")) != EOF) {
        switch (ch) {
            case 'h':
                db_home_dir = optarg;
                break;
            case '?':
            default:
                return (usage());
        }
    }
}
```

Далее создается дескриптор базы данных и определяются флаги открытия окружения. Здесь следует отметить несколько моментов:

- **DB_RECOVER** означает, что каждый раз, когда мы запускается приложение, запускается обычное восстановление. Это крайне желательно и рекомендуется для большинства приложений.
- также указывается **DB_THREAD**, что означает, что дескриптор окружения будет совместно использоваться потоками (free-threaded).

```

/* Create the environment */
ret = db_env_create(&envp, 0);
if (ret != 0) {
    fprintf(stderr, "Ошибка создания дескриптора окружения: %s\n",
               db_strerror(ret));
    goto err;
}
env_flags =
    DB_CREATE |           // Создать окружение, если не существует
    DB_INIT_LOCK |       // Инициализация блокировок
    DB_INIT_LOG |        // Инициализация журналирования
    DB_INIT_MPOOL |      // Инициализация кеширования в памяти
    DB_THREAD |          // Окружение будет использоваться совместно
    DB_INIT_TXN |         // Инициализация транзакционной подсистемы.
                        // Это также включает журналирование
    DB_RECOVER;         // Run normal recovery

```

Теперь настройка обнаружения взаимоблокировок, проходя по своим внутренним таблицам блокировок в поисках блока каждый раз, когда запрашивается блокировка.

Кроме того, в случае взаимоблокировки поток, удерживающий самую молодую блокировку, получит уведомление о взаимоблокировке.

Примечание

Каждая операция с базой данных проверяет код возврата статуса операции, и если возвращается статус ошибки (ненулевой), выполняется переход к метке `err`.

В отличие от объектно-ориентированных программ, таких как C++ или Java, в языке C нет блоков try. Поэтому это лучший способ реализовать каскадную обработку ошибок для данного примера.

```
// Указываем, чтобы db выполняла обнаружение блокировок внутренним образом.  
// Также указываем, что уведомление о взаимоблокировке в соответствующем  
// случае получит транзакция с наименьшим количеством блокировок записи.  
ret = envp->set_lk_detect(envp, DB_LOCK_MINWRITE);  
if (ret != 0) {  
    fprintf(stderr, "Error setting lock detect: %s\n", db_strerror(ret));  
    goto err;  
}
```

Теперь откроем окружение.

```
// Если бы у нас были служебные потоки (например, для запуска контрольных  
// точек или обнаружения взаимоблокировок), мы бы создали их здесь. Однако  
// для простого примера, такого как этот, это не требуется.  
  
// Теперь открываем окружение  
ret = envp->open(envp, db_home_dir, env_flags, 0);  
if (ret != 0) {  
    fprintf(stderr, "Ошибка открытия окружения: %s\n", db_strerror(ret));  
    goto err;  
}
```

Теперь мы вызываем функцию, которая откроет для нас базу данных.

Это не очень интересно, за исключением того, что вы заметите, что мы указываем `DB_DUPSORT`, что требуется исключительно для данных, которые мы записываем в базу данных, и это необходимо только в том случае, если приложение запускается более одного раза без предварительного удаления окружения.

Кроме того, никакого журнала ошибок здесь не предоставляется, поскольку функция `open_db( )`, описанная далее, делает это сама.

```
ret = open_db(&dbp, prog_name, file_name, envp, DB_DUPSORT);
if (ret != 0) {
    goto err;
}
```

Теперь мы создаем потоки, используя pthreads. Мы просто инициализируем мьютекс, создаем потоки, а затем их присоединяем, что заставляет программу ждать, пока присоединенные потоки не завершатся, прежде чем продолжить операции в основном потоке.

```
// Инициализация мьютекса. Используется для идентификации потоков.
(void)pthread_mutex_init(&thread_num_lock, NULL);

/* Start the writer threads. */
for (i = 0; i < NUMWRITERS; i++) {
    (void)pthread_create(&writer_threads[i], NULL, // атриб. по умолчанию
                        writer_thread, (void *)dbp);
}
```

```
/* Join the writers */
for (i = 0; i < NUMWRITERS; i++) {
    (void)pthread_join(writer_threads[i], NULL);
}
```

Наконец, завершая `main( )`, мы закрываем базу данных и дескриптор окружения, как это обычно бывает в любом приложении БД.

Обратите внимание, что именно здесь в приложении размещается метка `err`.

Если какая-либо операция с базой данных до этой точки в программе возвращает статус ошибки, программа просто переходит к этой точке и закрывает дескрипторы, если это необходимо, перед тем как полностью выйти из приложения.

```
err:
    // закрываем дескриптор БД, если он открыт
    if (dbp != NULL) {
        ret_t = dbp->close(dbp, 0);
        if (ret_t != 0) {
            fprintf(stderr, "%s database close failed: %s\n",
                    file_name, db_strerror(ret_t));
            ret = ret_t;
        }
    }
}
```



```
// закрываем окружение, если открыто
if (envp != NULL) {
    ret_t = envp->close(envp, 0);
    if (ret_t != 0) {
        fprintf(stderr, "environment close failed: %s\n",
            db_strerror(ret_t));
        ret = ret_t;
    }
}

// Выводим сообщение перед выходом и выходим
printf("I'm all done.\n");
return (ret == 0 ? EXIT_SUCCESS : EXIT_FAILURE);
}
```

Теперь, когда мы завершили `main( )`, нам нужно реализовать функцию, которую потоки-писатели будут фактически запускать.

Именно здесь находится большая часть транзакционного кода.

Как обычно, все начинается с объявления переменных и инициализации.

```
/*
Функция, выполняющая серию записей в базу данных Berkeley DB. Записываемая в
базу данных информация по большей части бессмысленна, но здесь
проиллюстрированы механизмы фиксации/отмены транзакций и обнаружения
взаимоблокировок.
*/
void* writer_thread(void *args) {

    static char *key_strings[] = {
        "key 1", "key 2", "key 3", "key 4", "key 5",
        "key 6", "key 7", "key 8", "key 9", "key 10"
    };
    DB      *dbp;
    DB_ENV *envp;
    DBT      key;
    DBT      value;
    DB_TXN *txn;
    int i, j, payload, ret, thread_num;
    int retry_count;
    int max_retries = 20; // Максимальное кол-во повторов в случае тупика

    dbp  = (DB *)args;          // получим дескриптор базы данных из параметров
    envp = dbp->get_env(dbp);    // получим дескриптор окружения для базы данных
```

Теперь нам нужен номер потока для целей отчетности. Можно использовать значение `pthread_t` напрямую для этой цели, но то, как это делается, к сожалению, отличается в зависимости от реализации `pthread`. Поэтому вместо него используется защищенная мьютексом глобальная переменная целого типа.

Также этот идентификатор потока используется для инициализации генератора случайных чисел, который генерирует данные.

```
// Получим номер своего потока
(void)pthread_mutex_lock(&thread_num_lock);
global_thread_num++;
thread_num = global_thread_num;
(void)pthread_mutex_unlock(&thread_num_lock);
// Инициализация генератора случайных чисел
srand(thread_num);
```

Теперь мы начинаем цикл, который мы используем для записи данных в БД.

В начале цикла верхнего уровня мы начинаем новую транзакцию.

Фактически будет реализовано 50 транзакций на поток записи, хотя одновременно в потоке будет только одна активная транзакция. В каждой транзакции выполняется 10 записей в базу данных.

Объединяя несколько записей вместе в одной транзакции, мы увеличиваем вероятность возникновения взаимоблокировки. Обычно есть желание уменьшить потенциал взаимоблокировки, и в этом случае простой способ это сделать — выполнять одну запись на транзакцию.

Чтобы избежать взаимоблокировок, можно было использовать автоматическую фиксацию для записи в базу данных для этой рабочей нагрузки.

Однако задача примера — показать обработку взаимоблокировок, и выполняя несколько записей на транзакцию, мы можем фактически наблюдать возникновение взаимоблокировок. Также пример демонстрирует идею о том, что можно объединить несколько операций с базой данных вместе в одну атомарную единицу работы, чтобы повысить эффективность записей.

Наконец, в вопросе стиля можно заметить метку **retry:**, которая размещена непосредственно перед кодом начала транзакции. Она используется для организации цикла в случае обнаружения взаимоблокировки и необходимости выполнения операции записи.

Очень многим не нравится цикл с операторами **goto**, и, конечно, можно было написать код, чтобы этого избежать. Однако, как мы увидим, использование **goto** в этом случае значительно помогает прояснить код, поэтому заблуждение Никлауса Вирта против использования **goto** игнорируем и организуем цикл обработки условия ошибки явно.



```
// Выполняем 50 раз операцию записи и выходим
for (i = 0; i < 50; i++) {
    retry_count = 0; // Используется для повторных попыток
    // Некоторые считают дурным тоном использовать оператор goto в цикле,
    // но мы всё равно делаем именно так, потому что это самый простой и
    // понятный способ реализовать операцию прерывания/повторения.
    retry: // внешний цикл обнаружения тупика и повторения попытки записи
    // Начинаем нашу транзакцию. Мы группируем несколько записей в этом
    // потоке в одну транзакцию, чтобы (1) показать, что можно атомарно
    // выполнять несколько записей одновременно, и (2) повысить вероятность
    // возникновения взаимоблокировки, чтобы обеспечить её обнаружение.

    // Обычно мы хотим избежать потенциальных взаимоблокировок, поэтому для
    // этой нагрузки правильным решением было бы выполнять put-коммиты с
    // помощью autocommit. Но это слишком упростило бы наш пример, поэтому
    // мы делаем «неправильный» вариант.

    ret = envp->txn_begin(envp, NULL, &txn, 0);
    if (ret != 0) {
        envp->err(envp, ret, "txn_begin failed");
        return ((void *)EXIT_FAILURE);
    }
}
```

Теперь мы начинаем внутренний цикл, который мы используем для фактического выполнения записи. Мы используем оператор switch-case для проверки кода возврата из метода put. Этот оператор используется для определения того, нужно ли прерывать (или прерывать и повторять в случае взаимоблокировки) текущую транзакцию.

```
for (j = 0; j < 10; j++) {  
    // устанавливаем ключи  
    memset(&key, 0, sizeof(DBT));  
    key.data = key_strings[j];  
    key.size = (u_int32_t)strlen(key_strings[j]) + 1;  
    // устанавливаем значения  
    memset(&value, 0, sizeof(DBT));  
    payload = rand() + i;  
    value.data = &payload;  
    value.size = sizeof(int);  
    // Выполняем метод put  
    switch (ret = dbp->put(dbp, txn, &key, &value, 0)) {  
    case 0:  
        break;  
        // База данных настроена на сортировку дубликатов, поэтому  
        // существует вероятность возврата ошибки KEYEXIST.  
        // Если мы получим такую ошибку, ее следует игнорировать и  
        // продолжать работу.  
        // !!! Ошибки KEYEXIST можно увидеть только после того, как  
        // программа отработает хотя бы один раз.
```

```
case DB_KEYEXIST:
    printf("Got keyexists.\n");
    break;
    // Здесь мы выполняем обнаружение взаимоблокировки.
    // Если операция put возвращает DB_LOCK_DEADLOCK, то этот поток
    // был выбран для разрешения взаимоблокировки.
    // Он должен прервать операцию и, при необходимости, повторить
    // попытку put.
case DB_LOCK_DEADLOCK:
    // Первое, что мы ДОЛЖНЫ сделать, это отменить транзакцию.
    (void)txn->abort(txn);
    // Теперь решаем, нужно ли повторять операцию. Если количество
    // повторных попыток меньше max_retries, увеличиваем значение
    // счетчика повторов и переходим к повтору.
    if (retry_count < max_retries) {
        printf("Writer %i: Got DB_LOCK_DEADLOCK.\n", thread_num);
        printf("Writer %i: Retrying write operation.\n", thread_num);
        retry_count++;
        goto retry;
    }
    // В противном случае просто сдаемся
    printf("Writer %i: ", thread_num);
    printf("Got DB_LOCK_DEADLOCK and out of retries.\n");
    printf("Writer %i: Giving up.\n", thread_num);
    return ((void *)EXIT_FAILURE);
```

```

        // Если возникает общая ошибка, мы просто прерываем транзакцию
        // и полностью выходим из потока.
default:
    envp->err(envp, ret, "db put failed");
    ret = txn->abort(txn);
    if (ret != 0) envp->err(envp, ret, "txn abort failed");
    return ((void *)EXIT_FAILURE);
} // End switch-case statement
} // End for loop

```

Завершив внутренний цикл записи базы данных, мы могли бы просто зафиксировать транзакцию и перейти к следующему блоку из 10 записей. Однако мы хотим сначала проиллюстрировать несколько моментов относительно обработки транзакций, поэтому вместо этого мы вызываем функцию `count_records( )` перед вызовом фиксации транзакции. Функция `count_records( )` использует курсор для чтения каждой записи в базе данных и возвращает количество записей, которые она нашла.

```

    printf("Thread %i. Records: %i\n", thread_num, count_records(dbp, NULL));
    // фиксируем транзакцию
    ret = txn->commit(txn, 0);
    if (ret != 0) {
        envp->err(envp, ret, "txn commit failed");
        return ((void *)EXIT_FAILURE);
    }
    return ((void *)EXIT_SUCCESS);
}

```


Второй параметр функции `count_records( )` принимает дескриптор транзакции. Однако в данном случае дескриптор транзакции в `count_records( )` не передается.

Поскольку `count_records()` считывает каждую запись в базе данных, при неправильном использовании поток самоблокируется. Поток записи только что записал 500 записей в базу данных, но поскольку транзакция, используемая для этой записи, еще не была зафиксирована, каждая из этих 500 записей все еще заблокирована транзакцией потока.

Если мы затем просто запустим нетранзакционный курсор в базе данных из того же потока, который заблокировал эти 500 записей, курсор заблокируется, когда попытается прочитать одну из этих транзакционно защищенных записей. Поток немедленно останавливает работу в этой точке, пока курсор ждет запрошенной им блокировки чтения. Поскольку эта блокировка чтения никогда не будет снята (поток никогда не сможет двигаться вперед), это представляет собой самоблокировку для потока.

Есть три способа предотвратить эту самоблокировку:

1) Мы можем переместить вызов `count_records( )` в точку после того, как транзакция потока будет зафиксирована.

2) Мы можем разрешить `count_records( )` работать в той же транзакции, в которой были выполнены все записи (именно для этого и нужен параметр `transaction` для функции).

3) Мы можем уменьшить гарантию изоляции для приложения, разрешив незафиксированные чтения.

Для этого примера мы выбираем использовать вариант 3 (незафиксированные чтения), чтобы избежать взаимоблокировки. Это означает, что мы должны открыть базу данных так, чтобы она поддерживала незафиксированные чтения, и мы должны открыть дескриптор курсора, чтобы он знал, что нужно выполнять незафиксированные чтения.

Ниже представлена реализация `count_records()`. В этой функции нет ничего особенно интересного, кроме указания незафиксированных чтений при открытии дескриптора курсора, но мы в любом случае включаем функцию сюда для полноты.

```
// Эта функция просто подсчитывает количество записей в базе данных и  
// возвращает результат. Эту функцию можно использовать тремя способами:  
// 1) вызывая ее с активным дескриптором txn.  
// 2) настроив курсор для незафиксированных чтений (то, что сделано в примере).  
// 3) вызвав count_records ПОСЛЕ того, как писатель зафиксировал транзакцию.  
// Если не сделать ничего из этого, поток сообщений зайдет в тупик.  
// Эта функция здесь только для наглядности. Более простой способ подсчёта  
// количества записей в базе данных – функция DB->stat() или DB->stat_print().
```

```
int count_records(DB *dbp, DB_TXN *txn) {  
  
    DBT key, value;  
    DBC *cursorp;  
    int count, ret;  
    cursorp = NULL;  
    count = 0;  
  
    /* Get the cursor */  
    ret = dbp->cursor(dbp, txn, &cursorp, DB_READ_UNCOMMITTED);  
    if (ret != 0) {  
        dbp->err(dbp, ret, "count_records: cursor open failed.");  
        goto cursor_err;  
    }  
}
```

```
// получаем key DBT для чтения базы данных
```

```
memset(&key, 0, sizeof(DBT));
```

```
memset(&value, 0, sizeof(DBT));
```

```
do {
```

```
    ret = cursorp->get(cursorp, &key, &value, DB_NEXT);
```

```
    switch (ret) {
```

```
        case 0: // все в порядке
```

```
            count++;
```

```
            break;
```

```
        case DB_NOTFOUND:
```

```
            break;
```

```
        default:
```

```
            dbp->err(dbp, ret, "Count records unspecified error");
```

```
            goto cursor_err;
```

```
    }
```

```
} while (ret == 0);
```

```
cursor_err:
```

```
    if (cursorp != NULL) {
```

```
        ret = cursorp->close(cursorp);
```

```
        if (ret != 0) {
```

```
            dbp->err(dbp, ret, "count_records: cursor close failed.");
```

```
        }
```

```
    }
```

```
    return (count);
```

```
}
```

Наконец, реализация функции `open_db( )`.

Следует обратить внимание, что когда открывается база данных, указываются незафиксированные чтения. Если этого не сделать, то функция `count_records( )` привела бы поток к самоблокировке, поскольку курсор не мог бы быть открыт для поддержки незафиксированных чтений (этот флаг при открытии курсора, по сути, молча игнорировался бы базой данных).

```
// Open a Berkeley DB database
int open_db(DB **dbpp,
            const char *progname,
            const char *file_name,
            DB_ENV *envp,
            u_int32_t extra_flags) {

    int ret;
    u_int32_t open_flags;
    DB *dbp;

    // инициализация дескриптора
    ret = db_create(&dbp, envp, 0);
    if (ret != 0) {
        fprintf(stderr, "%s: %s\n", progname, db_strerror(ret));
        return (EXIT_FAILURE);
    }
}
```

```

// выносим полученный дескриптор наружу db_create()
*dbpp = dbp;

if (extra_flags != 0) {
    ret = dbp->set_flags(dbp, extra_flags);
    if (ret != 0) {
        dbp->err(dbp, ret, "open_db: Attempt to set extra flags failed.");
        return (EXIT_FAILURE);
    }
}
// открываем базу данных
open_flags = DB_CREATE      | //
    DB_READ_UNCOMMITTED    | // грязные чтения
    DB_AUTO_COMMIT         | // автофиксация
    DB_THREAD;             // совместное использование потоками
ret = dbp->open(dbp,        /* Pointer to the database */
    NULL,                 /* Txn pointer */
    file_name,            /* File name */
    NULL,                 /* Logical db name */
    DB_BTREE,             /* Database type (using btree) */
    open_flags,           /* Open flags */
    0);                   /* File mode. Using defaults */
if (ret != 0) {
    dbp->err(dbp, ret, "Database '%s' open failed", file_name);
    return (EXIT_FAILURE);
}
return (EXIT_SUCCESS);
}

```

Пример транзакции в памяти

Иногда БД используется для приложений, которым просто нужно кэшировать данные, полученные из другого места (например, удаленного сервера баз данных). БД также часто используется во встроенных системах. В обоих случаях приложения могут захотеть использовать транзакции для атомарности, согласованности и гарантий изоляции, но они также могут захотеть полностью отказаться от гарантии долговечности. При этом они могут хранить свое окружение БД и базы данных полностью в памяти, чтобы избежать влияния на производительность ненужного дискового ввода-вывода.

Для этого:

- Воздержитесь от указания домашнего каталога при открытии окружения. Исключением является случай, когда вы используете файл конфигурации `DB_CONFIG` — в этом случае следует указать домашний каталог окружения, чтобы можно было найти файл конфигурации.
- Настройте окружение для резервного копирования областей из системной памяти, а не из файловой системы.
- Настройте подсистему ведения журнала таким образом, чтобы файлы журналов хранились полностью в памяти.
- Увеличьте размер буфера журнала в памяти, чтобы он был достаточно большим для хранения самого большого набора одновременных операций записи.
- Увеличьте размер кэша в памяти, чтобы он мог хранить весь набор данных. Вы не хотите, чтобы кэш выгружался на диск.
- Не указывайте имя файла при открытии базы данных.

В качестве примера в этом разделе рассматривается пример транзакции, представленный в Примере транзакции, стр. 86), и он обновляет этот пример так, чтобы окружение, ба-

за данных, файлы журналов и области полностью хранились в памяти.

Для целей иллюстрации мы также изменяем этот пример так, чтобы незафиксированные чтения больше не использовались для включения функции `count_records()`. Вместо этого мы просто предоставляем дескриптор транзакции для `count_records()`, чтобы избежать самоблокировки. Имейте в виду, что использование здесь дескриптора транзакции вместо незафиксированных чтений будет работать так же хорошо, как если бы мы продолжали использовать незафиксированные чтения. Однако использование дескриптора транзакции здесь, вероятно, вызовет больше взаимоблокировок, чем использование `read-uncommitted`, поскольку в этом случае выполняется больше блокировок.

Для начала немного упростим начало примера. Поскольку нам больше не нужен домашний каталог окружения, мы можем удалить весь код, который мы использовали для определения разделителей пути и включить функцию `getopt`. Мы также можем удалить функцию `usage()`, поскольку нам больше не требуются никакие аргументы командной строки.

```
/* File: txn_guide_inmemory.c */
#define _DEFAULT_SOURCE // Для -std=c11
/* We assume an ANSI-compatible compiler */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <db.h>
#include <unistd.h>

/* Run 5 writers threads at a time. */
```

```

#define NUMWRITERS 5

// Printing of a thread_t is implementation-specific, so we
// create our own thread IDs for reporting purposes.

int global_thread_num;
pthread_mutex_t thread_num_lock;

// Forward declarations
int count_records(DB *, DB_TXN *);
int open_db(DB **, const char *, const char *, DB_ENV *, u_int32_t);
void* writer_thread(void *);

int main(void) {

    /* Initialize our handles */
    DB *dbp      = NULL;
    DB_ENV *envp = NULL;

    pthread_t writer_threads[NUMWRITERS];
    int i, ret, ret_t;
    u_int32_t env_flags;

    /* Application name */
    const char *prog_name = "txn_guide_inmemory";

```

Далее мы создаем окружение, как обычно. Однако мы добавляем DB_PRIVATE к фла-

гам открытия окружения. Этот флаг заставляет окружение резервировать области, используя память кучи приложения, а не файловую систему. Это первый важный шаг к сохранению данных БД полностью в памяти.

Мы также удаляем флаг DB_RECOVER из флагов открытия окружения. Поскольку базы данных, журналы и области хранятся в памяти, восстанавливать ничего не придется.

Обратите внимание, что мы выделяем здесь дополнительный код жирным шрифтом.

```
/* Create the environment */
ret = db_env_create(&envp, 0);
if (ret != 0) {
    fprintf(stderr, "Ошибка создания дескриптора окружения: %s\n",
              db_strerror(ret));
    goto err;
}
env_flags =
    DB_CREATE |           // Создать окружение, если не существует
    DB_INIT_LOCK |       // Инициализация блокировок
    DB_INIT_LOG |        // Инициализация журналирования
    DB_INIT_MPOOL |      // Инициализация кеширования в памяти
    DB_THREAD |          // Free-thread the env handle
    DB_INIT_TXN |        // Инициализация транзакций
    DB_PRIVATE;          // Region files are not backed by the filesystem.
                        // Instead, they are backed by heap memory.
```

Теперь мы настраиваем окружение для хранения файлов журнала в памяти, увеличиваем размер буфера журнала до 10 МБ и увеличиваем кэш в памяти до 10 МБ. Этих значений

должно быть более чем достаточно для рабочей нагрузки приложения.

```
// Specify in-memory logging
ret = envp->log_set_config(envp, DB_LOG_IN_MEMORY, 1);
if (ret != 0) {
    fprintf(stderr, "Error setting log subsystem to in-memory: %s\n",
            db_strerror(ret));
    goto err;
}

// Specify the size of the in-memory log buffer.
ret = envp->set_lg_bsize(envp, 10 * 1024 * 1024);
if (ret != 0) {
    fprintf(stderr, "Error increasing the log buffer size: %s\n",
            db_strerror(ret));
    goto err;
}

// Specify the size of the in-memory cache.
ret = envp->set_cachesize(envp, 0, 10 * 1024 * 1024, 1);
if (ret != 0) {
    fprintf(stderr, "Error increasing the cache size: %s\n",
            db_strerror(ret));
    goto err;
}
```

Далее мы открываем окружение и настраиваем обнаружение блокировки. Это идентич-

но тому, как работал предыдущий пример, за исключением того, что мы не указываем местоположение для домашнего каталога окружения.

```
// Indicate that we want db to perform lock detection internally.
// Also indicate that the transaction with the fewest number of write
// locks will receive the deadlock notification in the event of a deadlock.
ret = envp->set_lk_detect(envp, DB_LOCK_MINWRITE);
if (ret != 0) {
    fprintf(stderr, "Error setting lock detect: %s\n",
            db_strerror(ret));
    goto err;
}

// Now actually open the environment
ret = envp->open(envp, NULL, env_flags, 0);
if (ret != 0) {
    fprintf(stderr, "Ошибка открытия окружения: %s\n",
            db_strerror(ret));
    goto err;
}
```

Когда мы вызываем `open_db()`, который мы используем для открытия базы данных, мы не предоставляем имя файла базы данных для третьего параметра. Когда имя файла равно `NULL`, база данных не поддерживается файловой системой.

```
// If we had utility threads (for running checkpoints or
// deadlock detection, for example) we would spawn those
```

```
// here. However, for a simple example such as this,  
// that is not required.
```

```
// Open the database  
ret = open_db(&dbp, prog_name, NULL,  
             envp, DB_DUPSORT);  
if (ret != 0) {  
    goto err;  
}
```

После этого функция main() остается неизменной, за исключением того, что при закрытии базы данных мы изменяем строку сообщения об ошибке, чтобы не ссылаться на имя файла базы данных.

```
// Initialize a mutex. Used to help provide thread ids  
(void)pthread_mutex_init(&thread_num_lock, NULL);  
  
// Start the writer threads  
for (i = 0; i < NUMWRITERS; i++) {  
    (void)pthread_create(  
        &writer_threads[i], NULL, writer_thread, (void *)dbp);  
}  
  
// Join the writers  
for (i = 0; i < NUMWRITERS; i++) (void)pthread_join(writer_threads[i],  
NULL);
```

```
err:
    // Close our database handle, if it was opened.
    if (dbp != NULL) {
        ret_t = dbp->close(dbp, 0);
        if (ret_t != 0) {
            fprintf(stderr, "%s database close failed.\n",
                    db_strerror(ret_t));
            ret = ret_t;
        }
    }

    // Close our environment, if it was opened.
    if (envp != NULL) {
        ret_t = envp->close(envp, 0);
        if (ret_t != 0) {
            fprintf(stderr, "environment close failed: %s\n",
                    db_strerror(ret_t));
            ret = ret_t;
        }
    }

    // Final status message and return.
    printf("I'm all done.\n");
    return (ret == 0 ? EXIT_SUCCESS : EXIT_FAILURE);
}
```

Это завершает `main()`. Основная часть реализации функции `writer_thread()` не изменилась по сравнению с первоначальным примером транзакции, за исключением того, что мы больше не проверяем `DB_KEYEXISTS` в коде возврата `DB->put()`. Поскольку мы настраиваем полностью in-memory базу данных, нет никакой возможности запустить этот код для существующей базы данных. Следовательно, нет способа, при котором `DB->put()` вернет `DB_KEYEXISTS`.

```
// A function that performs a series of writes to a
// Berkeley DB database. The information written
// to the database is largely nonsensical, but the
// mechanism of transactional commit/abort and
// deadlock detection is illustrated here.
void* writer_thread(void *args) {

    static char *key_strings[] = {
        "key 1", "key 2", "key 3", "key 4", "key 5",
        "key 6", "key 7", "key 8", "key 9", "key 10"
    };
    DB      *dbp;
    DB_ENV *envp;
    DBT key, value;
    DB_TXN *txn;
    int i, j, payload, ret, thread_num;
    int retry_count, max_retries = 20;    /* Max retry on a deadlock */

    dbp = (DB *)args;
```

```
envp = dbp->get_env(dbp);

/* Get the thread number */
(void)pthread_mutex_lock(&thread_num_lock);
global_thread_num++;
thread_num = global_thread_num;
(void)pthread_mutex_unlock(&thread_num_lock);

/* Initialize the random number generator */
srand(thread_num);

/* Write 50 times and then quit */
for (i = 0; i < 50; i++) {
    retry_count = 0; /* Used for deadlock retries */

    // Some think it is bad form to loop with a goto statement, but
    // we do it anyway because it is the simplest and clearest way
    // to achieve our abort/retry operation.
retry:
    // Begin our transaction. We group multiple writes in
    // this thread under a single transaction so as to
    // (1) show that you can atomically perform multiple writes
    // at a time, and (2) to increase the chances of a
    // deadlock occurring so that we can observe our
    // deadlock detection at work.
    //
    // Normally we would want to avoid the potential for deadlocks,
```

```

// so for this workload the correct thing would be to perform our
// puts with autocommit. But that would excessively simplify our
// example, so we do the "wrong" thing here instead.
ret = envp->txn_begin(envp, NULL, &txn, 0);
if (ret != 0) {
    envp->err(envp, ret, "txn_begin failed");
    return ((void *)EXIT_FAILURE);
}
for (j = 0; j < 10; j++) {
    /* Set up our key and values DBTs */
    memset(&key, 0, sizeof(DBT));
    key.data = key_strings[j];
    key.size = (u_int32_t)strlen(key_strings[j]) + 1;

    memset(&value, 0, sizeof(DBT));
    payload = rand() + i;
    value.data = &payload;
    value.size = sizeof(int);

    /* Perform the database put. */
    switch (ret = dbp->put(dbp, txn, &key, &value, 0)) {
    case 0:
        break;

        // Here's where we perform deadlock detection. If
        // DB_LOCK_DEADLOCK is returned by the put operation,
        // then this thread has been chosen to break a deadlock.
    }
}

```



```

        // It must abort its operation, and optionally retry the
        // put.
case DB_LOCK_DEADLOCK:
    // First thing that we MUST do is abort the
    // transaction.
    (void)txn->abort(txn);
    // Now we decide if we want to retry the operation.
    // If we have retried less than max_retries,
    // increment the retry count and goto retry.
    if (retry_count < max_retries) {
        printf("Writer %i: Got DB_LOCK_DEADLOCK.\n",
            thread_num);
        printf("Writer %i: Retrying write operation.\n",
            thread_num);
        retry_count++;
        goto retry;
    }
    /*
     * Otherwise, just give up.
     */
    printf("Writer %i: ", thread_num);
    printf("Got DB_LOCK_DEADLOCK and out of retries.\n");
    printf("Writer %i: Giving up.\n", thread_num);
    return ((void *)EXIT_FAILURE);
    /*
     * If a generic error occurs, we simply abort the
     * transaction and exit the thread completely.

```

```

        */
    default:
        envp->err(envp, ret, "db put failed");
        ret = txn->abort(txn);
        if (ret != 0) envp->err(envp, ret, "txn abort failed");
        return ((void *)EXIT_FAILURE);
    } /** End case statement */

} /** End for loop */

```

Единственное другое изменение в `writer_thread()` заключается в том, что мы передаем `count_records()` дескриптор транзакции, а не настраиваем все приложение для незафиксированных чтений. Оба механизма работают достаточно хорошо для предотвращения самоблокировки. Однако индивидуальный счетчик в этом примере будет, как правило, ниже счетчиков, показанных в предыдущем примере транзакции, поскольку `count_records()` больше не может видеть записи, созданные, но еще не зафиксированные другими потоками.

```

// print the number of records found in the database.
// See count_records() for usage information.
printf("Thread %i. Record count: %i\n", thread_num,
        count_records(dbp, txn));

/*
 * If all goes well, we can commit the transaction and
 * exit the thread.
 */

```

```
    ret = txn->commit(txn, 0);
    if (ret != 0) {
        envp->err(envp, ret, "txn commit failed");
        return ((void *)EXIT_FAILURE);
    }
}
return ((void *)EXIT_SUCCESS);
}
```

Далее мы обновляем `count_records()`. Единственное отличие здесь в том, что мы больше не указываем `DB_READ_UNCOMMITTED` при открытии курсора. Обратите внимание, что даже это незначительное изменение не требуется. Если мы не настроим базу данных для поддержки незафиксированных чтений, `DB_READ_UNCOMMITTED` при открытии курсора будет молча игнорироваться. Однако мы все равно убираем флаг из открытия курсора, чтобы избежать путаницы.

```
// This simply counts the number of records contained in the
// database and returns the result. You can use this function
// in three ways:
//
// First call it with an active txn handle (this is what the
// example currently does).
//
// Secondly, configure the cursor for uncommitted reads.
//
// Third, call count_records AFTER the writer has committed
```

```
//     its transaction.
//
// If you do none of these things, the writer thread will
// self-deadlock.
//
// Note that this function exists only for illustrative purposes.
// A more straight-forward way to count the number of records in
// a database is to use DB->stat() or DB->stat_print().

int count_records(DB *dbp, DB_TXN *txn) {

    DBT key, value;
    DBC *cursorp;
    int count, ret;

    cursorp = NULL;
    count    = 0;

    // Get the cursor
    ret = dbp->cursor(dbp, txn, &cursorp, 0);
    if (ret != 0) {
        dbp->err(dbp, ret,
                "count_records: cursor open failed.");
        goto cursor_err;
    }

    // Get the key DBT used for the database read
```

```

memset(&key, 0, sizeof(DBT));
memset(&value, 0, sizeof(DBT));
do {
    ret = cursorp->get(cursorp, &key, &value, DB_NEXT);
    switch (ret) {
        case 0:
            count++;
            break;
        case DB_NOTFOUND:
            break;
        default:
            dbp->err(dbp, ret,
                    "Count records unspecified error");
            goto cursor_err;
    }
} while (ret == 0);

cursor_err:
    if (cursorp != NULL) {
        ret = cursorp->close(cursorp);
        if (ret != 0) {
            dbp->err(dbp, ret,
                    "count_records: cursor close failed.");
        }
    }
    return (count);
}

```

Наконец, мы обновляем `open_db()`. Это включает удаление `DB_READ_UNCOMMITTED` из флагов открытия. Мы также внимательно изменяем сообщение об ошибке открытия базы данных, чтобы больше не использовать переменную `file_name`, поскольку это значение всегда будет `NULL` для этого примера.

```
// Open a Berkeley DB database
int open_db(DB **dbpp,
            const char *progrname,
            const char *file_name,
            DB_ENV *envp,
            u_int32_t extra_flags) {

    int ret;
    u_int32_t open_flags;
    DB *dbp;

    // Initialize the DB handle
    ret = db_create(&dbp, envp, 0);
    if (ret != 0) {
        fprintf(stderr, "%s: %s\n", progrname,
                db_strerror(ret));
        return (EXIT_FAILURE);
    }

    // Point to the memory malloc'd by db_create()
    *dbpp = dbp;
```

```

if (extra_flags != 0) {
    ret = dbp->set_flags(dbp, extra_flags);
    if (ret != 0) {
        dbp->err(dbp, ret,
            "open_db: Attempt to set extra flags failed.");
        return (EXIT_FAILURE);
    }
}

// Now open the database
open_flags =
    DB_CREATE          | // Разрешаем создать базу данных
    DB_THREAD          |
    DB_AUTO_COMMIT;    // Разрешаем автофиксацию (auto-commit)

ret = dbp->open(dbp,          /* Pointer to the database */
               NULL,         /* Txn pointer */
               file_name,    /* File name */
               NULL,        /* Logical db name */
               DB_BTREE,     /* Database type (using btree) */
               open_flags,   /* Open flags */
               0);           /* File mode. Using defaults */

if (ret != 0) {
    dbp->err(dbp, ret, "Database open failed");
    return (EXIT_FAILURE);
}

```

```
}    return (EXIT_SUCCESS);
```